

SERVICE WEB

Séance 1 - Introduction aux architectures web

Cours 25604 - Collège de Maisonneuve

Web Services - HTTP - REST - SOAP - GraphQL - Node.js - React - Next.js

Plan de la séance

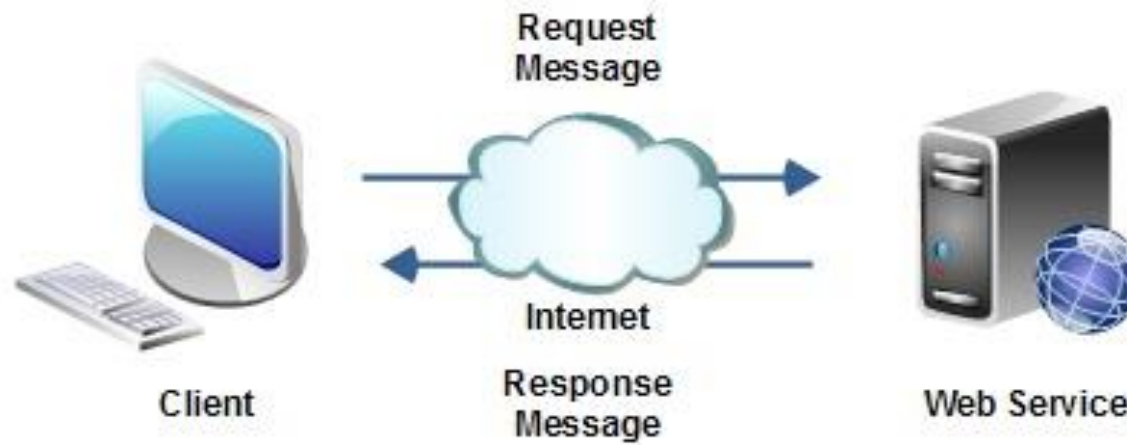
Ce que vous allez apprendre aujourd'hui

- **1. Qu'est-ce qu'un service Web ?** - Site, application, service
- **2. Architecture client-serveur** - Le protocole HTTP en détail
- **3. Requêtes HTTP** - Méthodes, URL, en-têtes, corps
- **4. Réponses HTTP** - Codes de statut (2xx, 3xx, 4xx, 5xx)
- **5. Les types d'API** - SOAP, REST, GraphQL
- **6. Formats d'échange** - JSON, XML, YAML, CSV
- **7. Sérialisation / Désérialisation** - `JSON.stringify()` / `JSON.parse()`
- **8. Installation du backend Node.js** - Premier serveur HTTP
- **9. Frontend React** - Composants, JSX, props
- **10. Framework Next.js** - API Routes + Mini-projet Blog

Qu'est-ce qu'un service Web ?

Définition

Un service web est une interface logicielle qui permet à deux applications de communiquer entre elles via Internet, en utilisant le protocole HTTP (ou HTTPS) pour échanger des données structurées (souvent JSON ou XML).



En clair : un service web est comme un pont numérique entre deux programmes.

L'un demande une information (le client), l'autre répond (le serveur), sans intervention humaine.

Site Web vs Application Web vs Service Web

Trois notions souvent confondues

1. SITE WEB

- Pages destinées aux humains
- Fonction informative
- Contenu statique ou peu dynamique
- Exemples : Wikipédia, portfolio, site vitrine
- **Tech** : HTML, CSS, peu de JS
- **But** : informer et communiquer



2. APPLICATION WEB

- Système interactif (actions concrètes)
- Logique métier + frontend + backend
- Données dynamiques et personnalisées
- Exemples : Gmail, Facebook, Amazon
- **Tech** : HTML/CSS/JS + Node.js/PHP/Python
- **But** : permettre des interactions en ligne



3. SERVICE WEB (API)

- S'adresse à d'autres applications
- Ne renvoie pas de HTML mais du JSON / XML
- Standardisé via HTTP / HTTPS
- Exemples : Stripe, OpenWeather, Google Maps
- **Tech** : HTTP, JSON, REST, SOAP, GraphQL
- **But** : faire communiquer des systèmes



Exemple concret - API météo

Comment ton téléphone reçoit la météo

Ton application météo appelle l'API publique Open-Meteo :

HTTP

GET https://api.open-meteo.com/v1/forecast?lat=45.5&lon=-73.6

HTTP

Le service répond avec du JSON (données brutes lisibles par les programmes) :

```
{
  "latitude": 45.5,
  "longitude": -73.566,
  "timezone": "America/Toronto",
  "hourly": {
    "temperature_2m": [8.2, 7.6, 7.1],
    "relative_humidity_2m": [85, 87, 89]
  }
}
```

JSON

Dictionnaire

Résultat

- Code de statut : 200 OK
- Format : application/json
- L'app lit ces données
- Affiche : 8.2 °C / 85 % humidité

Le service web ne montre rien à l'écran : il fournit les données à une autre application.

Pourquoi les services Web sont essentiels aujourd'hui

Le langage commun du monde numérique

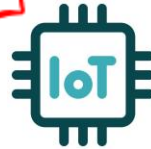
Sans services web, les applications modernes ne pourraient pas coopérer.

Communication entre apps

- CRM - ERP - e-commerce - marketing
- Échanges entre systèmes hétérogènes
- Tout passe par HTTP + JSON

Internet des objets (IoT)

- Thermostat intelligent
- Montre connectée
- Capteurs industriels
- Communication via API REST ou MQTT



Intégrations ouvertes

- Google Maps API
- YouTube API - OpenAI API
- Slack / Discord bots
- Stripe pour les paiements . AWT



Automatisation

- Webhooks (notifications)
- Pipelines de données
- Bots et assistants
- CI/CD et déploiement

↓
Vercel

« Les services web sont la colonne vertébrale invisible

qui connecte les systèmes, les objets, les utilisateurs et les entreprises. »

Plus ton service web est ouvert -> plus ton projet devient puissant.

Architecture client-serveur & protocole HTTP

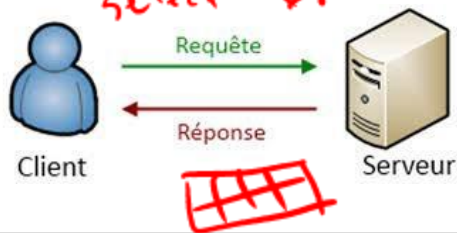
Rappel fondamental

- **Client** : demande une ressource ou une action (navigateur, app mobile)
- **Serveur** : reçoit la requête, traite (logique + BD), répond
- **Stateless (REST)** : chaque requête contient tout le contexte nécessaire

Types d'architecture client-serveur :

Architecture 2 tiers

- 1 client + 1 serveur
- Toutes les ressources sur un seul serveur
- Simple, idéal pour petits projets
- Exemple : site avec BD intégrée



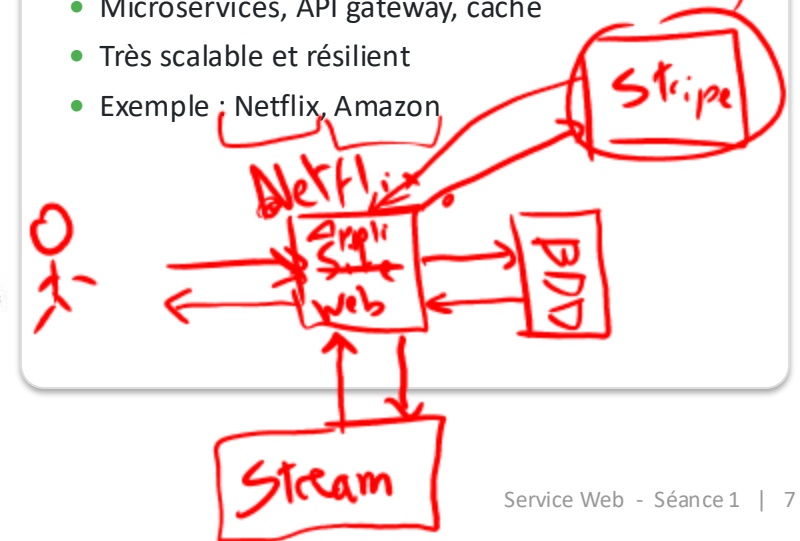
Architecture 3 tiers

- 1 client + 1 serveur app + 1 serveur BD
- Séparation logique et stockage
- Plus robuste, plus scalable
- Standard de la plupart des web apps



Architecture n tiers

- Plus de 3 acteurs
- Microservices, API gateway, cache
- Très scalable et résilient
- Exemple : Netflix, Amazon



La requête HTTP - structure générale

Ligne de requête + en-têtes + corps (optionnel)

<METHODE> <CHEMIN>?<QUERY> HTTP/1.1

HTTP

Une URL complète a la forme :

https://api.exemple.com:443/users/42?expand=profile&lang=fr

URL

Élément	Rôle	Exemple
Schéma	Protocole utilisé	http ou https
Hôte	Nom de domaine ou IP du serveur	api.exemple.com
Port	Par défaut 80 (HTTP) ou 443 (HTTPS)	:443
Chemin (path)	Désigne la ressource demandée	/users/42
Query	Données optionnelles (filtres, pagination)	?expand=profile&lang=fr

Les méthodes HTTP (verbes)

GET, POST, PUT, PATCH, DELETE

La méthode indique ce que le client veut faire sur la ressource ciblée :



Méthode	Utilisation	Exemple	Avec un corps ?
GET ✓	Lire des données	GET /users -> liste des utilisateurs <i>Select * From Users</i>	Non
POST ✓	Créer une ressource	POST /users -> créer un utilisateur <i>Insert 1 into Users</i>	Oui (JSON) <i>Attribut</i>
PUT ✓	Remplacer entièrement	PUT /users/10 -> écrase l'ancien <i>id</i>	Oui (JSON) ✓
PATCH ✓	Modifier partiellement	PATCH /users/10 -> change le nom	Oui (JSON) ✓
DELETE ✓	Supprimer une ressource	DELETE /users/10 -> <i>id=10 User</i>	Non

Règle simple :

- GET et DELETE : sans corps (~~en général~~)
- POST, PUT, PATCH : avec un corps JSON contenant les données

nom	Age	login	sexe
○	○	○	○

Les en-têtes (headers)

Métadonnées de la requête

Paires clé:valeur situées juste après la ligne de requête, avant le corps.

```
GET /api/users?page=1 HTTP/1.1
Host: api.exemple.com
Accept: application/json
Authorization: Bearer abc123xyz
User-Agent: Mozilla/5.0 (Windows NT 10.0)
```

HTTP

→ Headers

→ Corps

Généraux

- Host, User-Agent, Connection, Cache-Control

Contenu

- Content-Type : application/json ✓
- Content-Length : taille du corps ✓

Acceptation

- Accept : application/json, text/html ✓
- Accept-Language : fr-CA, fr; q=0.8
- Accept-Encoding : gzip, deflate, br

Autorisation / sécurité

- Authorization : Bearer <token> ✓
- Cookie : envoi de session navigateur
- Origin : indique la source (CORS) ✓
- X-API-Key : 123456 (custom) ✗

Le corps de la requête (body)

Les données envoyées au serveur

Utilisé avec les méthodes POST, PUT, PATCH. Souvent au format JSON.

Format	Content-Type	Utilisation
JSON	application/json	Standard moderne (API REST)
Formulaire	application/x-www-form-urlencoded	Formulaire HTML
Fichier	multipart/form-data	Upload (images, PDF...)
XML	application/xml	Systèmes anciens / SOAP

Banque

Exemple JSON (API REST) :

POST /api/patients HTTP/1.1

Host: api.hopital.com

Content-Type: application/json

HTTP

```
{ "name": "Borris Ulysse", "birthDate": "1998-06-12", "gender": "male", "phone": "+15148872107" }
```

Exemple complet - requête & réponse

Une transaction HTTP de bout en bout

Requête envoyée par le client :

```
POST /api/users HTTP/1.1
Host: api.cmaisonneuve.org
Authorization: Bearer eyJhbGc...
Content-Type: application/json
Accept: application/json
Accept-Language: fr-CA, fr; q=0.9
Connection: keep-alive

{
  "name": "Ziane Massinissa",
  "email": "ziane@cmaisonneuve.ca",
  "role": "student", note: "LL"
}
```

REQUEST

Réponse renvoyée par le serveur :

```
HTTP/1.1 201 Created
Content-Type: application/json
Location: /api/users/56

{ "id": 56, "name": "Ziane Massinissa" }
```

RESPONSE

email:
role:
id = UUID - coid

Ce que fait le serveur :

- Lit la ligne de requête (POST /api/users)
- Vérifie les en-têtes (JSON, token, langue)
- Décode le corps JSON et crée l'utilisateur
- Renvoie 201 Created + Location + données

Les codes de statut HTTP

5 catégories : 1xx, 2xx, 3xx, 4xx, 5xx

2xx - Succès

- 200 OK -> tout s'est bien passé *Get*
- 201 Created -> ressource créée (POST)
- 202 Accepted -> en cours (asynchrone) •
- 204 No Content -> succès sans corps (DELETE, PUT)

3xx - Redirections

- 301 Moved Permanently -> redirection permanente
- 302 Found -> redirection temporaire
- 304 Not Modified -> utiliser le cache

4xx - Erreurs client

- 400 Bad Request -> requête mal formée
- 401 Unauthorized -> token manquant
- 403 Forbidden -> accès refusé
- 404 Not Found -> ressource inexistante •
- 422 Unprocessable Entity -> erreur de validation

5xx - Erreurs serveur

- 500 Internal Server Error -> bug, crash
- 502 Bad Gateway -> mauvaise réponse upstream
- 503 Service Unavailable -> surcharge, maintenance *.Drops*
- 504 Gateway Timeout -> lenteur backend

La réponse HTTP - corps & en-têtes

Ce que renvoie le serveur

Une réponse est composée de : ligne de statut + en-têtes + corps

Exemple - corps JSON (API REST) :

```
{
  "id": 42,
  "name": "Eddine Rabah",
  "email": "eddine@cmaisonneuve.org",
  "role": "Teacher"
}
```

JSON

Exemple - corps HTML (site web) :

```
<!DOCTYPE html>
<html>
  <head><title>Bienvenue</title></head>
  <body><h1>Bonjour Eddine !</h1></body>
</html>
```

HTML

En-têtes de réponse principaux :

En-tête	Rôle
Content-Type	Type de données (application/json, text/html, image/png)
Content-Length	Taille du corps en octets
Cache-Control	Règles de mise en cache (no-cache, max-age=3600)
ETag	Identifiant unique pour valider le cache
Set-Cookie	Envoi d'un cookie au navigateur

Qu'est-ce qu'une API ?

Application Programming Interface

Une API est un pont de communication entre deux programmes. Elle permet à un système A (ton frontend, ton téléphone) de demander un service à un système B (serveur, BD, app externe).

Analogie : au restaurant

Toi = CLIENT

- Ton application (frontend ou mobile)
- Tu passes une commande
- Tu ne parles pas au cuisinier

Le serveur = API

- Reçoit la commande
- Transmet au format attendu
- Rapporte la réponse de la cuisine

Cuisine = BACKEND

- Système derrière l'API
- Base de données, logique métier
- Prépare la réponse

Mini exercice à essayer dans le terminal :

```
curl -s "https://api.adviceslip.com/advice" | jq
```

bash

SOAP - Simple Object Access Protocol

Protocole formel (1990s) - Microsoft, IBM

Protocole formel basé sur XML, utilisé dans les systèmes bancaires, médicaux et gouvernementaux.

Repose sur :

- HTTP (ou SMTP, JMS)
- Format unique : XML
- WSDL (contrat du service)
- WS-Security, WS-Addressing

```
<soap:Envelope xmlns:soap="...">
  <soap:Body>
    <getPatient>
      <id>123</id>
    </getPatient>
  </soap:Body>
</soap:Envelope>
```

XML

Avantages

- Standardisé (WSDL, WS-Security)
- Fortement typé et contractuel
- Sécurisé (signature, chiffrement)
- Indépendant du protocole

Inconvénients

- Verbeux (XML lourd)
- Complexe à configurer
- Lent (parsing XML)
- Peu flexible pour les applis modernes

REST - Representational State Transfer

Roy Fielding, 2000 - le standard moderne

REST n'est pas un protocole mais un style d'architecture. Chaque ressource est identifiée par une URL unique et manipulée avec les verbes HTTP.

Action	Méthode	Exemple d'URL	Corps attendu
Lire	GET	/api/patients/123	—
Créer	POST	/api/patients	{ "name": "Ahmed" }
Mettre à jour	PUT / PATCH	/api/patients/123	{ "name": "Ahmed B." }
Supprimer	DELETE	/api/patients/123	—

Avantages

- Simple, lisible, basé sur HTTP
- Rapide et léger (JSON)
- Facile à tester (Postman, curl)
- Standard de facto (mobile, web)

Inconvénients

- Overfetching / underfetching
- Pas de typage fort
- Multiplication des endpoints
- Caching parfois limité

GraphQL - Graph Query Language

Facebook, 2015 - une seule URL, une requête précise

Le client spécifie exactement les données dont il a besoin. Une seule URL, une seule requête, une réponse adaptée.

```
query {  
  patient(id: 123) {  
    name  
    birthDate  
    doctor {  
      name  
      specialty  
    }  
  }  
}
```

GraphQL

```
{  
  "data": {  
    "patient": {  
      "name": "Maxime Ulysse",  
      "birthDate": "1998-06-12",  
      "doctor": {  
        "name": "Dr Francois",  
        "specialty": "Cardiologie" } } } } }
```

JSON

Avantages

- Une seule requête = toutes les données
- Pas d'overfetching / underfetching
- Typage fort via schéma SDL
- Excellent pour React, Next.js

Inconvénients

- Mise en cache plus complexe
- Courbe d'apprentissage
- Requêtes coûteuses possibles
- Débogage plus technique

Formats d'échange - JSON, XML, YAML, CSV

Choisir le bon format selon le contexte

JSON (le plus utilisé sur le web)

```
{  
  "name": "Sonia",  
  "age": 27,  
  "skills": ["Web", "Python"],  
  "student": true  
}
```

JSON

XML (utilisé par SOAP)

```
<user>  
  <name>Sonia</name>  
  <age>27</age>  
  <skills>  
    <skill>Web</skill>  
  </skills>  
</user>
```

XML

YAML (config : Docker, K8s, CI)

```
name: Eddine  
age: 27  
skills:  
  - Web  
  - Python  
teacher: true
```

YAML

CSV (tabulaire : data science)

```
name,age,teacher  
Eddine,27,true  
Amine,25,false
```

CSV

JSON = format standard pour les API REST et GraphQL (léger, lisible, natif en JavaScript)

Sérialisation & Désérialisation

Convertir un objet JS en JSON et inversement

Pour transporter un objet sur le réseau, il faut le transformer en texte JSON.

Objet JS

JSON.stringify()

"chaîne JSON"

JSON.parse()

Sérialisation (objet -> texte) :

```
const user = { name: "Eddine", age: 27 };  
const jsonData = JSON.stringify(user);  
// {"name":"Eddine","age":27}
```

JS

Désérialisation (texte -> objet) :

```
const json = '{"name":"Eddine","age":27}';  
const user = JSON.parse(json);  
console.log(user.name); // Eddine
```

JS

Exemple concret avec fetch() :

```
fetch("/api/users", { method: "POST",  
  headers: { "Content-Type": "application/json" },  
  body: JSON.stringify(user) // sérialisation  
}).then(r => r.json()) // désérialisation auto
```

JS

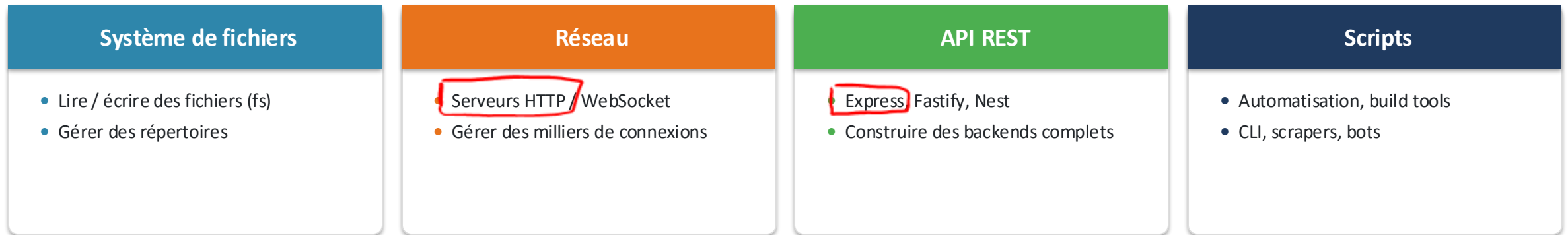
PARTIE 2 - BACKEND

Installation & premier serveur Node.js

Introduction à Node.js

JavaScript côté serveur

Avant Node.js, JavaScript ne s'exécutait que dans les navigateurs. Grâce à Node.js, JS est devenu un langage full-stack capable d'héberger des serveurs, manipuler des fichiers, accéder au réseau...



Architecture Single-Thread, Non-Blocking

Contrairement à Java ou PHP (multithread), Node n'utilise qu'un seul thread principal. Mais grâce à son Event Loop, il peut gérer des milliers de connexions en parallèle sans se bloquer.

```
console.log("1 - Début");  
setTimeout(() => console.log("2 - Tâche asynchrone après 2s"), 2000);  
console.log("3 - Fin immédiate");
```

JS

INSTALLATION - Node.js & npm

Étape par étape (Windows / Mac / Linux)

Étape 1 - Télécharger Node.js (version LTS) :

`https://nodejs.org` -> Télécharger la version LTS -> Installer

WEB

Étape 2 - Vérifier l'installation dans le terminal :

```
node -v
npm -v
```

bash

```
v22.18.0
10.9.3
```

output

Étape 3 - Comprendre npm (Node Package Manager) :

Commande	Description
<code>npm init -y</code>	Crée un nouveau projet Node (avec package.json)
<code>npm install express</code>	Installe la librairie Express (serveur web)
<code>npm install axios</code>	Installe axios pour faire des requêtes HTTP
<code>npm start</code>	Lance le projet (défini dans package.json)
<code>npm list</code>	Liste les dépendances installées

Packages npm essentiels pour le backend

À installer dans votre projet Node.js

Pour démarrer un projet backend professionnel, voici les packages les plus utilisés :

Serveur web

- **express** -> framework web minimaliste
- **cors** -> autoriser le partage cross-origin *connexion BE-FE*
- helmet -> en-têtes de sécurité
- morgan -> logger HTTP

Données & base de données

- mongoose -> ORM pour MongoDB
- pg / mysql2 -> drivers SQL
- **prisma** -> ORM moderne (SQL)
- **dotenv** -> charger les variables .env

Sécurité & auth

- bcrypt -> hashage de mots de passe ✓
- jsonwebtoken -> tokens JWT ✓
- express-validator -> validation des entrées ✓
- axios -> requêtes HTTP sortantes ✓

Développement

- **nodemon** -> redémarre auto le serveur
- **typescript** -> typage statique
- jest -> tests unitaires
- eslint -> linter de code

Démarrer un projet backend Node.js

Les commandes complètes pas à pas

1. Créer un nouveau dossier projet et initialiser :

```
mkdir mon-api  
cd mon-api  
npm init -y
```

bash

2. Installer les dépendances de base (Express + outils dev) :

```
npm install express cors dotenv  
npm install --save-dev nodemon
```

bash

3. Ajouter un script de démarrage dans package.json :

```
"scripts": {  
  "start": "node app.js",  
  "dev": "nodemon app.js"  
}
```

JSON

4. Lancer en mode développement (auto-restart) : npm run dev

Premier serveur Node.js (sans framework)

Le module http natif

1. Créer le projet :

```
mkdir serveur-bienvenue  
cd serveur-bienvenue  
npm init -y
```

bash

2. Créer app.js :

```
const http = require('http');  
  
const server = http.createServer((req, res) => {  
  res.writeHead(200, { "Content-Type": "text/html" });  
  res.end("<h1>Bienvenue sur mon premier serveur Node.js !</h1>");  
});  
  
server.listen(8080, () => console.log("Serveur actif sur http://localhost:8080"));
```

JS

3. Lancer le serveur, puis ouvrir le navigateur :

```
node app.js  
-> http://localhost:8080
```

bash

Serveur Express - exemple complet

Le framework le plus utilisé en Node.js

Installation :

```
npm install express
```

bash

Fichier app.js :

```
const express = require('express');
const app = express();
app.use(express.json());      // parse le JSON automatiquement

// Route GET -> lire des données
app.get('/api/users', (req, res) => {
  res.json([ { id: 1, name: "Alex" }, { id: 2, name: "Julie" } ]);
});

// Route POST -> créer une ressource
app.post('/api/users', (req, res) => {
  const newUser = req.body;
  res.status(201).json({ message: "Utilisateur créé", user: newUser });
});

app.listen(3000, () => console.log("API sur http://localhost:3000"));
```

JS

PARTIE 3 - FRONTEND

React + Next.js

Introduction à React.js

Bibliothèque JavaScript de Facebook pour construire des UI

Pourquoi on aime React :

Avantage	Explication
Composants réutilisables	Le code est découpé en petits blocs indépendants
Performance élevée	React met à jour uniquement ce qui change (Virtual DOM)
JSX	Écrire du HTML directement dans du JavaScript
SPA fluide	Pas de rechargement de page, tout est dynamique
Écosystème immense	Next.js, React Native, Redux...

Installation et démarrage (prérequis : Node.js + npm) :

```
npx create-react-app mon-premier-react
cd mon-premier-react
npm start           # ouvre http://localhost:3000
```

bash

Composants React - JSX + props

Le bloc de base de toute application React

Composant simple :

```
function Bonjour() {  
  return <h1>Bonjour tout le monde !</h1>;  
}  
  
export default Bonjour;
```

JSX

Composant avec props :

```
function Bonjour(props) {  
  return <h2>Salut {props.nom} !</h2>;  
}  
  
// <Bonjour nom="Sonia" />
```

JSX

Structure typique d'un projet React :

```
mon-premier-react/  
  node_modules/      -> dépendances (à ne jamais modifier)  
  public/  
    index.html        -> page principale unique  
  src/  
    App.js            -> composant principal  
    index.js          -> point d'entrée du projet  
    components/       -> tes propres composants  
    package.json      -> configuration du projet
```

tree

Next.js - framework React full-stack

React + Routing + SSR + API backend intégré

Next.js est un framework basé sur React qui permet de créer des applications web complètes (frontend + backend) avec rendu côté serveur (SSR), génération statique (SSG), routing automatique et API Routes intégrées.

Rendu SSR / SSG

- Côté serveur ou statique
- Excellent pour le SEO

Routing auto

- Basé sur le système de fichiers
- Dossier app/ -> routes

API Routes

- Backend dans le projet
- Plus besoin d'Express séparé

Optimisations

- Images, fonts, scripts
- Build ultra rapide

Installation et démarrage en 1 minute :

```
npx create-next-app@latest my-app  
cd my-app  
npm run dev           # http://localhost:3000
```

bash

Le setup par défaut peut activer TypeScript, Tailwind, App Router et Turbopack.

API Routes dans Next.js

Backend intégré, sans serveur externe

Une API Route est un endpoint HTTP que l'on définit directement dans notre projet Next.js. Plus besoin d'Express, Flask ou Spring Boot.

Version Next.js	Dossier	Fichier typique	URL d'accès
Avant Next 13 (Pages Router)	/pages/api/	/pages/api/users.js	/api/users
Depuis Next 13+ (App Router)	/app/api/	/app/api/users/route.ts	/api/users

Exemple - `app/api/users/route.ts` :

```
import { NextResponse } from "next/server";

export async function GET() {
  const users = [{ id: 1, name: "Alex" }, { id: 2, name: "Julie" }];
  return NextResponse.json(users);
}

export async function POST(req: Request) {
  const body = await req.json();
  return NextResponse.json({ message: `Utilisateur ${body.name} ajouté` });
}
```

TS

Exemple complet - Mini Blog Next.js

Backend (API) + Frontend dans un seul projet

Backend - app/api/posts/route.ts :

```
import { NextResponse } from "next/server";

let posts = [
  { id: 1, title: "Intro Next.js", content: "..." },
  { id: 2, title: "Server Components", content: "..." }
];

export async function GET() {
  return NextResponse.json(posts);
}

export async function POST(req: Request) {
  const newPost = await req.json();
  posts.push({ id: posts.length + 1, ...newPost });
  return NextResponse.json({ message: "Ajouté !" });
}
```

TS

Frontend - app/page.tsx :

```
"use client";
import { useEffect, useState } from "react";

export default function Home() {
  const [posts, setPosts] = useState([]);

  useEffect(() => {
    fetch("/api/posts")
      .then(res => res.json())
      .then(data => setPosts(data));
  }, []);

  return (
    <main>
      <h1>Mini Blog</h1>
      {posts.map(p => (
        <div key={p.id}>
          <h2>{p.title}</h2>
          <p>{p.content}</p>
        </div>))}
    </main>
  );
}
```

TSX

Récapitulatif - ce qu'il faut retenir

Synthèse de la séance 1

Concepts clés

- Service web = pont entre 2 apps
- HTTP = protocole client-serveur
- Méthodes : GET, POST, PUT, DELETE
- Statuts : 2xx, 3xx, 4xx, 5xx
- Formats : JSON > XML

Types d'API

- SOAP -> protocole XML, banques
- REST -> standard, JSON, simple
- GraphQL -> 1 requête, données ciblées
- Choisir selon le contexte

Stack moderne

- Backend : Node.js + Express
- Frontend : React (composants)
- Full-stack : Next.js (App Router)
- API Routes Next = backend intégré

Commandes d'installation essentielles à mémoriser :

```
# Vérifier Node
node -v && npm -v

# Backend Express
mkdir mon-api && cd mon-api && npm init -y
npm install express cors dotenv

# Frontend Next.js (full-stack)
npx create-next-app@latest my-app && cd my-app && npm run dev
```

bash



Questions ?

Préparez votre environnement pour la prochaine séance :

Node.js (LTS) + VS Code + Postman + navigateur

